PyDataUI Developer Cookbook (v1.2.1)

> **Production Full-Stack Python Framework for Data & AI Applications**
> Build reactive web apps and interactive terminal dashboards in pure Python without writing JavaScript or dealing with top-to-bottom script reruns.

Table of Contents

1. The PyDataUI Mental Model
2. Recipe 1: Zero-Class Procedural State
3. Recipe 2: Production AI Chatbot with Loading Indicators
4. Recipe 3: Universal Tabular Data Sniffing (DuckDB, Polars, Pandas)
5. Recipe 4: Pythonic Spatial Layouts (Row, Col, Grid, Spacer)
6. Recipe 5: Dynamic Forms from Pydantic Models
7. Recipe 6: Multi-Page Directory Routing (`pages/`)
8. Recipe 7: Dual-Surface Terminal UI (TUI) Mode
9. Recipe 8: FastAPI Native Mounting & Microservices
10. Recipe 9: Production Deployment & Dockerization

---

1. The PyDataUI Mental Model

Why PyDataUI Over Streamlit?

Streamlit re-executes your entire `.py` script from line 1 on every single user click. This creates heavy caching workarounds (`@st.cache_data`) and consumes high RAM per session.

PyDataUI operates on standard ASGI (Starlette / FastHTML / HTMX): 1. **Event Delegation**: Clicking a button or sending a message sends a lightweight HTMX POST event (`/_pydataui/event`). 2. **Targeted Mutation**: Only the targeted `@s.action` runs. 3. **DOM Diff Swapping**: Only the affected component subtree is rendered and swapped into the DOM. 4. **Dual Surface**: The exact same code runs in a modern web browser and as an interactive Terminal UI (TUI).

```
from pydataui import App, state, Button, Text, Row

app = App()
s = state(count=0)

@s.action
def increment():
    s.count += 1

@app.page("/")
def home():
    return Row(
        Button("-", on_click=lambda: s.set(count=s.count - 1)),
        Text(s.count),
        Button("+", on_click=increment)
    )

if __name__ == "__main__":
    app.run()
```

Recipe 1: Zero-Class Procedural State

Eliminate verbose classes for rapid prototypes while retaining full reactivity.

```
from pydataui import App, state, Input, Button, Text, Card, Col

app = App(title="Procedural State")

# 1. Initialize state with defaults
s = state(
    query="",
    results=[],
    status="Ready"
)

# 2. Define action handlers
@s.action
def search(query: str = ""):
    val = query or s.query
    s.query = val
    s.status = "Searching..."
    # Inside actions, attributes resolve as native Python types:
    s.results = [f"Result for: {val} #{i}" for i in range(1, 4)]
    s.status = f"Found {len(s.results)} items."

@app.page("/")
def home():
    return Col(
        Input(placeholder="Search docs...", on_change=search),
        Text(s.status),
        Card(
            title="Search Results",
            children=[Text(r) for r in s.results]
        )
    )
```

Recipe 2: Production AI Chatbot with Loading Indicators & Modern UI/UX

Renders an interactive commercial-grade chat interface with: - **IME-Safe Multiline Textarea**: Auto-expanding with `Shift+Enter` for new line and `Enter` to submit. - **Built-in HTMX Indicators**: 3-dot pulsing typing bubble (`animate-bounce`), button spinner (`animate-spin`), and button/input double-submission locks (`hx-disabled-elt`). - **Clickable Suggestion Chips**: Auto-fills and triggers quick starter prompts with one click. - **Auto-Scroll to Bottom**: Smoothly keeps the newest messages in view on render and updates. - **Markdown & Code Formatting**: Beautiful syntax and styling for assistant responses with copy-to-clipboard button. - **Clear Chat Support**: Dedicated `on_clear` action handler and trash button in header.

```
import httpx
from pydataui import App, state, Container
from pydataui.components import Chat

app = App(title="AI Agent Chat")

s = state(
    messages=[
        {"role": "assistant", "content": "Hello! I am your AI Assistant. Ask me anything or sele
    ],
    session_id="chat_session_01",
    loading=False
)

@s.action
async def handle_message(message: str = "", **kwargs):
    user_msg = message or kwargs.get("message", "").strip()
    if not user_msg:
        return

    # Immediately push user message into history
    s.messages.append({"role": "user", "content": user_msg})
    s.loading = True

    try:
        # Non-blocking async API call to your LLM or LangGraph agent
        async with httpx.AsyncClient(timeout=120.0) as client:
            resp = await client.post(
                "http://127.0.0.1:8000/chat/invoke",
                json={"id": str(s.session_id), "message": user_msg}
            )
            resp.raise_for_status()
            reply = resp.json().get("response", "No response received.")
    except Exception as e:
        reply = f"Error: {e}"
    finally:
        s.loading = False

    s.messages.append({"role": "assistant", "content": reply})

@s.action
def clear_chat():
    s.messages = [
        {"role": "assistant", "content": "Chat history cleared. How can I assist you today?"}
    ]

@app.page("/")
def home():
    return Container(
        Chat(
            messages=s.messages,
            title="Customer Support AI",
            subtitle="SLM Agent • Model: Llama-3-8B • Context: 8k",
            placeholder="Type a message... (Shift+Enter for new line)",
            on_send=handle_message,
            on_clear=clear_chat,
            suggestions=[
                "How do I reset my password?",
                "What are your business hours?",
```

```
                "Show cluster compute telemetry"
            ],
            loading=s.loading,  # Triggers pulsing typing dots + button spinner
            height="650px",
            show_clear=True
        ),
        max_width="lg",
        padding="lg"
    )

if __name__ == "__main__":
    app.run(port=8000)
```

Recipe 3: Universal Tabular Data Sniffing

Pass tabular data directly into `DataTable` without manual conversion.

```
import polars as pl
import duckdb
import pandas as pd
from pydataui import App, DataTable, Card, Col, Heading

app = App(title="Data Sniffing")

# Polars
pl_df = pl.DataFrame({
    "model": ["Llama-3-8B", "Mistral-7B", "Phi-3-Mini"],
    "params_b": [8.03, 7.24, 3.82],
    "mmlu": [68.4, 62.7, 69.1]
})

# DuckDB Relation
duck_conn = duckdb.connect(":memory:")
duck_rel = duck_conn.sql("SELECT 'Worker-01' as node, 98.4 as load_pct UNION ALL SELECT 'Worker-(

# Pandas
pd_df = pd.DataFrame({
    "optimizer": ["AdamW", "Lion", "Sophia"],
    "lr": [0.0001, 0.00005, 0.0002]
})

@app.page("/")
def home():
    return Col(
        Heading("Universal Data Display", level=1),
        Card("Polars DataFrame", [DataTable(pl_df)]),
        Card("DuckDB PyRelation", [DataTable(duck_rel)]),
        Card("Pandas DataFrame", [DataTable(pd_df)])
    )
```

Recipe 4: Pythonic Spatial Layouts

Use declarative layout shorthands instead of raw Tailwind string hacking.

```
from pydataui import App, Row, Col, Grid, Spacer, Card, MetricCard, Text, Button

app = App()

@app.page("/")
def home():
    return Col(
        # Top KPI Metrics Row
        Row(
            MetricCard("Active Workers", 16, delta="+4 scaled"),
            MetricCard("Throughput", "2,450 req/s", delta="+14.2%"),
            MetricCard("Error Rate", "0.002%", delta="-0.01%"),
            gap="4"
        ),
        Spacer(size=8),
        # 3-Column Responsive Grid
        Grid(
            Card("Node Alpha", [Text("Status: Healthy")]),
            Card("Node Beta", [Text("Status: Training")]),
            Card("Node Gamma", [Text("Status: Idle")]),
            columns=3,
            gap="6"
        )
    )
```

Recipe 5: Dynamic Forms from Pydantic Models

Generate Shadcn-styled validated forms straight from Pydantic v2 classes.

```
from enum import Enum
from pydantic import BaseModel, Field
from pydataui import App, state, ShadForm, Container

class ComputeTier(str, Enum):
    STANDARD = "standard"
    ENTERPRISE = "enterprise"

class TrainingJob(BaseModel):
    experiment_name: str = Field(..., description="Unique job title")
    epochs: int = Field(10, description="Training epochs")
    learning_rate: float = Field(0.001, description="Learning rate")
    tier: ComputeTier = Field(ComputeTier.STANDARD, description="Compute tier")

app = App()
s = state(job_submitted=None)

@s.action
def launch_job(**kwargs):
    s.job_submitted = kwargs

@app.page("/")
def home():
    return Container(
        ShadForm.from_pydantic(
            TrainingJob,
            title="Launch SLM Job",
            on_submit=launch_job
        )
    )
```

Recipe 6: Multi-Page Directory Routing

Create a `pages/` directory next to your `app.py`. PyDataUI auto-discovers each page:

```
my_app/
├── app.py
└── pages/
    ├── 01_dashboard.py
    ├── 02_analytics.py
    └── 03_settings.py
```

Inside `pages/01_dashboard.py`:

```
from pydataui import Col, Heading, Text

TITLE = "Analytics Dashboard"
ICON = "chart-bar"
ORDER = 1
PUBLIC = True

def render():
    return Col(
        Heading("Analytics", level=1),
        Text("Real-time telemetry overview.")
    )
```

Recipe 7: Dual-Surface Terminal UI (TUI) Mode

Run your app without a browser in remote SSH, tmux, or Kubernetes pods:

```
# Launch interactive terminal UI
pydataui run --tui app.py

# Or single-shot ASCII render for cron logs
pydataui tui --once app.py
```

- When `Chat` is present, launches conversational terminal chat:
- `You > hello`
- `Customer Support AI is thinking...`
- `Customer Support AI: Hello! How can I assist you?`
- Single keypress `q` or `quit` exits immediately.

Recipe 8: FastAPI Native Mounting

Mount PyDataUI directly into existing FastAPI backends:

```
from fastapi import FastAPI
from pydataui import App, Container, Text, mount

api = FastAPI(title="Backend Microservice")
dashboard = App(title="Admin Panel")

@dashboard.page("/")
def admin_home():
    return Container(Text("Welcome to the Admin Panel"))

# API Route
@api.get("/api/v1/health")
def health():
    return {"status": "ok"}

# Mount PyDataUI at /dashboard
mount(api, dashboard, path="/dashboard")

# Run: uvicorn server:api --port 8000
```

Recipe 9: Production Deployment & Dockerization

Production Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["pydataui", "run", "app.py", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

Production Checklist

- [x] Configure reverse proxy (`nginx` or AWS ALB) to forward `X-Forwarded-For` and `X-Forwarded-Proto` .
- [x] Enable HTTPS for encrypted cookie transmission.
- [x] Run with multiple Uvicorn workers (`--workers 4`).
- [x] Pre-validate routes with `pydataui check app.py` .